

CSS

INTERVIEW REFERENCE

English-only • reviewed • interview-focused

A practical reference built from the supplied CSS interview/reference material, with accuracy checks and expanded revision coverage.

Core topics: selectors • specificity • box model • Flexbox • Grid • positioning • responsive design
• pseudo-classes • transitions • animations • variables • BEM • accessibility

Table of Contents

- 1. CSS Fundamentals
- 2. Selectors & Combinators
- 3. Cascade & Specificity
- 4. Colors & Backgrounds
- 5. Sizing & Units
- 6. Box Model
- 7. Display
- 8. Flexbox
- 9. CSS Grid
- 10. Positioning
- 11. z-index & Stacking Contexts
- 12. Overflow
- 13. Pseudo-classes & Pseudo-elements
- 14. Transitions & Transform
- 15. Animations
- 16. Responsive Design
- 17. CSS Variables & Functions
- 18. Typography
- 19. Borders & Shadows
- 20. Images & aspect-ratio
- 21. Forms & UI States
- 22. Inheritance
- 23. Modern Selectors
- 24. Logical Properties & Container Queries
- 25. BEM & CSS Architecture
- 26. Accessibility
- 27. CSS Performance
- 28. Common Pitfalls
- 29. Interview Q&A;
- 30. Practical Challenges
- 31. Rapid-fire Cheat Sheet
- 32. 7-Day Revision Plan

1. CSS Fundamentals

CSS (Cascading Style Sheets) controls the presentation and layout of HTML: color, spacing, typography, positioning, responsive layouts, and visual states.

Basic syntax:

```
p {\n  color: #333;\n  font-size: 16px;\n}
```

2. Selectors & Combinators

Selectors decide which elements receive a rule.

- `p` — element selector
- `.card` — class selector
- `#header` — ID selector
- `.card p` — descendant
- `.card > p` — direct child
- `h2 + p` — adjacent sibling
- `h2 ~ p` — general sibling
- `input[type="email"]` — attribute selector

3. Cascade & Specificity

Specificity is one part of the cascade used to resolve competing declarations. A useful interview model is: inline styles → IDs → classes/attributes/pseudo-classes → elements/pseudo-elements.

- If specificity is equal, the later matching rule normally wins.
- Modern cascade layers and origin/importance also matter, so 'ID always wins' is an oversimplification.
- `!important` changes cascade priority and should be used sparingly.

4. Colors & Backgrounds

Know color, background-color, background-image, gradients, background-size, background-position, and background-repeat.

```
.hero {\n  background: linear-gradient(135deg, #6c5ce7, #0984e3);\n  background-size: cover;\n  background-position: center;
```

5. Sizing & Units

Common units: px, %, rem, em, vh, vw, ch, and fr.

- rem is based on the root element's font size.
- em is context-dependent and can behave differently depending on the property.
- min-width/max-width constrain size.
- `clamp(min, preferred, max)` creates a bounded responsive value.

```
h1 { font-size: clamp(2rem, 5vw, 4rem); }
```

6. Box Model

The box model consists of content, padding, border, and margin.

- padding is inside the border; margin is outside.
- box-sizing: border-box includes padding and border in declared width/height.
- Vertical margins can collapse in normal block flow.

```
*, *::before, *::after { box-sizing: border-box; }
```

7. Display

Know block, inline, inline-block, flex, grid, and none.

- `display:none` removes the element from layout.
- `visibility:hidden` hides it while preserving its layout space.

8. Flexbox

Flexbox is mainly one-dimensional: items are arranged along a main axis and cross axis.

- `flex-direction` defines the main axis.
- `justify-content` works on the main axis.
- `align-items` works on the cross axis.
- `gap` adds spacing.
- `flex-wrap` allows multiple lines.
- Know `flex-grow`, `flex-shrink`, `flex-basis`, `align-self`, and `order`.

```
.container {\n  display:flex;\n  justify-content:center;\n  align-items:center;\n  min-height:100vh;\n}
```

9. CSS Grid

Grid is two-dimensional and handles rows and columns.

- Know `grid-template-columns`, `grid-template-rows`, `repeat()`, `gap`, `fr`, and `minmax()`.
- `auto-fit` and `auto-fill` are useful for responsive grids.

```
.cards {\n  display:grid;\n  grid-template-columns:repeat(auto-fit,minmax(220px,1fr));\n  gap:1rem;\n}
```

10. Positioning

Know static, relative, absolute, fixed, and sticky.

- `relative` stays in flow and is useful as a containing block for absolute descendants.
- `absolute` is removed from normal flow and positioned against a suitable containing block.
- `fixed` commonly stays attached to the viewport.
- `sticky` behaves relatively until a threshold, then sticks within its relevant scroll/containing context.

```
.card { position:relative; }\n.badge { position:absolute; top:8px; right:8px; }
```

11. z-index & Stacking Contexts

`z-index` controls stacking order within stacking contexts; it is not a single global stack.

- Flex and grid items can use `z-index` without traditional positioning.
- When `z-index` seems broken, inspect ancestor stacking contexts and properties that create them.

12. Overflow

Know visible, hidden, auto, `overflow-x`, and `overflow-y`.

- Ancestor `overflow/scroll` containers are a common cause of sticky-positioning surprises.

13. Pseudo-classes & Pseudo-elements

Pseudo-classes describe states/conditions; pseudo-elements style a generated or specific part.

- Pseudo-classes: `:hover`, `:focus`, `:focus-visible`, `:checked`, `:disabled`, `:nth-child()`, `:not()`.
- Pseudo-elements: `::before`, `::after`, `::first-letter`, `::selection`.

```
.title::after {\n  content:"";\n  display:block;\n  width:50px;\n  height:3px;\n}
```

14. Transitions & Transform

Transitions animate changes between states. Transforms can translate, rotate, scale, or skew without the same normal-flow layout change as width/margin.

- Know duration, timing function, delay, and transition-property.
- Common transforms: translate, scale, rotate, transform.

```
.button { transition:transform .2s ease, opacity .2s ease; }\n.button:hover { transform:translateY(-2px); opacity:.9; }
```

15. Animations

CSS animations use @keyframes to define stages.

- Know duration, timing function, delay, iteration count, direction, and fill mode.

```
@keyframes pulse {\n  from { transform:scale(1); }\n  to { transform:scale(1.05); }\n}\n\n.logo { animation:pulse 1.2s ease-in
```

16. Responsive Design

Responsive design adapts layout to viewport size and device conditions.

- Mobile-first means base styles target smaller screens and larger layouts are added progressively.
- Choose breakpoints where the content/design needs them; do not treat breakpoint numbers as universal laws.
- Use the viewport meta tag on responsive pages.

```
@media (min-width:768px) {\n  .cards { grid-template-columns:repeat(3,1fr); }\n}
```

17. CSS Variables & Functions

Custom properties improve reuse and theming.

- Declare with --name; read with var(--name).
- Know calc(), min(), max(), and clamp().

```
:root { --primary:#6c5ce7; --space:1rem; }\n.button { background:var(--primary); padding:var(--space); }
```

18. Typography

Know font-family, font-size, font-weight, line-height, text-align, letter-spacing, and word-spacing.

- Use readable line-height and reliable fallback fonts.

19. Borders, Radius & Shadows

Know border, border-radius, and box-shadow for cards, buttons, and surfaces.

```
.card { border:1px solid #ddd; border-radius:12px; box-shadow:0 8px 24px rgba(0,0,0,.08); }
```

20. Images & aspect-ratio

object-fit controls how replaced content such as images fits its box; aspect-ratio helps preserve a predictable shape.

- cover fills the box and can crop.
- contain keeps the full image and may leave empty space.
- object-position controls the focal position.

```
img { width:100%; aspect-ratio:4/3; object-fit:cover; object-position:center; }
```

21. Forms & UI States

Know focus, focus-visible, disabled, checked, required, valid, invalid, and placeholder-shown.

- Do not remove keyboard focus indicators without providing an equivalent.

22. Inheritance

Some properties naturally inherit; others do not.

- `inherit` uses the parent's computed value.
- `initial` uses the property's initial value.
- `unset` acts like `inherit` for inherited properties and `initial` for non-inherited properties.

23. Modern Selectors

Modern selectors include `:is()`, `:where()`, and `:has()`.

- `:is()` groups selectors.
- `:where()` groups selectors with zero specificity.
- `:has()` enables relational patterns based on matching descendants/relative selectors.

24. Logical Properties & Container Queries

Logical properties support writing-direction-aware layouts; container queries let components respond to their own container size.

- Examples: `margin-inline`, `padding-block`, `inset-inline-start`.

25. BEM & CSS Architecture

BEM means Block, Element, Modifier.

- Block: `card`
- Element: `card__title`
- Modifier: `card--featured`

```
<article class="card card--featured">\n  <h2 class="card__title">Product</h2>\n</article>
```

26. Accessibility

CSS should support keyboard users, readable contrast, and reduced-motion preferences.

- Keep visible focus.
- Do not rely on color alone.
- Respect `prefers-reduced-motion` when appropriate.
- Use semantic HTML; CSS cannot fix poor semantics.

```
@media (prefers-reduced-motion:reduce) {\n  * { animation-duration:.01ms; transition-duration:.01ms; }\n}
```

27. CSS Performance

At a high level: DOM/CSS information → style calculation → layout when needed → paint → compositing.

- Avoid unnecessary CSS and overly complex selectors.
- `Transform`/`opacity` are often good choices for animations, but measure real performance.
- Use `will-change` sparingly.

28. Common Pitfalls

Frequent interview mistakes:

- Overusing absolute positioning.

- Forgetting border-box.
- Thinking justify-content is always horizontal.
- Overusing !important.
- Treating z-index as a global stack.
- Using height:100% without a defined containing height.
- Forgetting min-width:0 on flex/grid children when content refuses to shrink.
- Assuming 100vh is always ideal on mobile.
- Using arbitrary breakpoints instead of content-driven ones.
- Breaking sticky behavior with unexpected scroll containers.

29. Interview Questions & Answers

Q: What is CSS?

A: A language for styling and laying out HTML documents.

Q: What is the box model?

A: Content, padding, border, and margin.

Q: What does border-box do?

A: Declared width/height include padding and border.

Q: Margin vs padding?

A: Margin is outside the border; padding is inside.

Q: Class vs ID?

A: Classes are reusable; IDs are intended to be unique and have higher specificity.

Q: What is specificity?

A: A cascade mechanism used to compare competing selectors.

Q: What does justify-content do?

A: Distributes flex/grid content along the relevant main/inline axis; in Flexbox it follows the main axis.

Q: What does align-items do?

A: Aligns flex items along the cross axis.

Q: Flexbox vs Grid?

A: Flexbox is mainly one-dimensional; Grid is two-dimensional.

Q: Fixed vs sticky?

A: Fixed commonly attaches to the viewport; sticky transitions from normal positioning to sticking within its scroll/containing context.

Q: What is z-index?

A: Stacking order inside stacking contexts.

Q: Does z-index always require position?

A: No. Flex/grid items can use z-index without traditional positioning.

Q: What is a transition?

A: An animation between two states.

Q: What are keyframes?

A: Named stages used by CSS animations.

Q: rem vs em?

A: rem uses the root font-size; em is context-dependent.

Q: display:none vs visibility:hidden?

A: The former removes layout participation; the latter preserves layout space.

Q: What is BEM?

A: Block, Element, Modifier naming convention.

Q: What is clamp()?

A: A bounded value: minimum, preferred, maximum.

Q: Why can sticky fail?

A: Wrong inset, insufficient scroll space, or ancestor scroll/overflow context are common causes.

Q: How do you make CSS responsive?

30. Practical Interview Challenges

Center a card

```
.page { min-height:100vh; display:grid; place-items:center; }
```

Responsive cards

```
.cards { display:grid; grid-template-columns:repeat(auto-fit,minmax(220px,1fr)); gap:1rem; }
```

Sticky navbar

```
.navbar { position:sticky; top:0; z-index:1000; }
```

Badge on card

```
.card { position:relative; }\n.badge { position:absolute; top:8px; right:8px; }
```

Hover button

```
.button { transition:transform .2s ease; }\n.button:hover { transform:translateY(-2px); }
```

Mobile-first layout

```
.nav { display:flex; flex-direction:column; }\n@media (min-width:768px) { .nav { flex-direction:row; } }
```

31. Rapid-fire Cheat Sheet

- Box model — content → padding → border → margin
- border-box — width/height include padding and border
- Flex — one-dimensional
- Grid — two-dimensional
- justify-content — main axis
- align-items — cross axis
- gap — space between items
- relative — in-flow + offsets / useful containing block
- absolute — removed from normal flow
- fixed — commonly viewport-attached
- sticky — sticks after threshold within its context
- z-index — stacking order within contexts
- :hover — pseudo-class
- ::before — pseudo-element
- transition — state change
- @keyframes — animation stages
- rem — root-based
- em — context-dependent
- clamp — bounded responsive value
- BEM — Block / Element / Modifier

32. 7-Day Revision Plan

Day 1 — Fundamentals, selectors, cascade, specificity, box model.

Day 2 — Display, Flexbox, axes, centering, wrapping, grow/shrink/basis.

Day 3 — Grid, responsive grids, positioning, fixed/sticky, z-index.

Day 4 — Pseudo-classes/elements, transitions, transform, animations.

Day 5 — Responsive design, units, clamp, variables, typography.

Day 6 — Forms, accessibility, BEM, modern selectors, logical properties, container queries.

Day 7 — Answer the Q&A; aloud and solve every practical challenge without looking.